Adaptive Resource Management and Quality Control for Streaming Video Generation

Yifei Xia, Hao Yuan, Suhan Ling, Haoran Sun, Hanke Zhang, Xupeng Miao, Fangcheng Fu, Bin Cui
The Hetu Team @ Peking University

Abstract

Autoregressive diffusion transformers (AR-DiTs) recast video generation from an offline paradigm to a real-time streaming one: the model generates video one chunk at a time, making each chunk available for playout once produced. The *service-level objective* (SLO) for this paradigm is no longer fixed latency or throughput, but *the preservation of playout continuity*: generation must stay ahead of the playout timeline. Once generation falls behind, the remaining playable buffer (*playout slack*) is exhausted, and users experience visible stalls. This objective reveals two serving design insights. First, real-time video generation has a dynamic SLO that evolves with playout progress, so resources should move toward streams with lower playout slack. Second, an acceptable chunk delivered on time is preferable to a late high-fidelity chunk, so per-chunk fidelity configurations should adapt to available playout slack. Guided by these insights, we present SlackServe, a playout-slack-driven serving system that preserves playout continuity in real-time streaming video generation. SlackServe uses playout slack as a unified signal, reallocating resources across streams through three-tier priority queues, re-homing, and elastic sequence parallelism, while selecting per-chunk fidelity configurations within each stream through *Bi-Modal Pareto Routing* under a quality floor. On a 16-H100 GPU cluster, SlackServe improves *Quality of Experience* (QoE), measured by *Continuous Play Ratio* (CPR), by 1.64×–3.29× and reduces *Time to First Chunk* (TTFC) by 1.61×–9.65× over baselines, while preserving comparable generation quality.

1 Introduction

Recent advances in video generation are pushing the field beyond offline, one-shot generation [28, 63, 71] toward autoregressive and interactive generation paradigms [20, 61, 76, 80]. Offline diffusion transformers (DiTs) [18, 51, 59], as shown in Figure 1(a) and (b), generate all frames through one multi-step denoising process over the entire video. This paradigm incurs high latency because denoising long video sequences is time-consuming, often taking tens of minutes to generate a 5-second, 81-frame video [69]. Thus, it can serve latency-insensitive scenarios where users can wait [9, 65], but falls short for interactive or real-time applications [26, 73].

Contact: Yifei Xia (yifeixia@stu.pku.edu.cn), Fangcheng Fu (ccchengff@sjtu.edu.cn) and Bin Cui (bin.cui@pku.edu.cn)

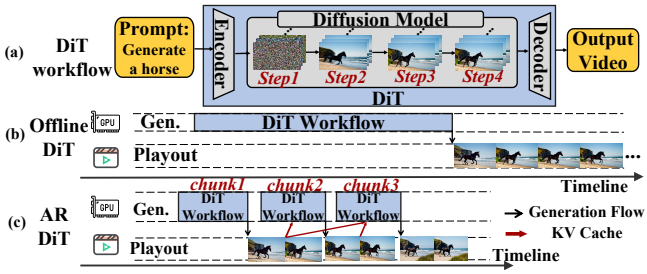

Figure 1. Workflow and generation paradigms of DiT-based video generation.

Autoregressive diffusion transformers (AR-DiTs) [12, 20, 36, 61, 76, 80], represented by Self-Forcing [20] and Causal-Forcing [80], recast video generation as a chunk-wise streaming process. As shown in Figure 1(c), the model generates video one chunk at a time, making each chunk available for playout once produced. For each chunk generation, the model uses the KV cache of previous chunks to preserve temporal coherence [15, 20]. By shortening the input sequence length, chunk-wise generation reduces generation time and, more importantly, makes real-time streaming possible [17, 26]. This capability enables emerging applications, such as AI live streaming and real-time visual effects [14, 27, 57].

Despite their promise, efficient serving for real-time streaming video generation remains a pressing yet underexplored system problem [14, 26, 27]. The core challenge is an objective mismatch: existing serving abstractions often center on fixed latency or throughput targets [1, 2, 29, 68, 75, 79], which are insufficient for this streaming setting. As shown in Figure 2, the primary *service-level objective* (SLO) [5] for each video stream is no longer a fixed latency target, such as *Time per Output Token* (TPOT) [79] in large language models [8, 62] (LLMs) or completion time in offline DiTs [68], but *preserving user-visible* ***playout continuity***. To be specific, the generated video must stay ahead of the playout timeline. If generation falls behind, the remaining playable buffer is exhausted, causing users to experience playout stalls. We define this remaining playable buffer as ***playout slack***.

This objective exposes two key serving properties that existing serving systems do not account for.

Property 1: Real-time video generation has a ***dynamic SLO*** *that evolves with the playout timeline.* As shown in Figure 2(c), unlike fixed TPOT-style targets in LLM serving, a real-time

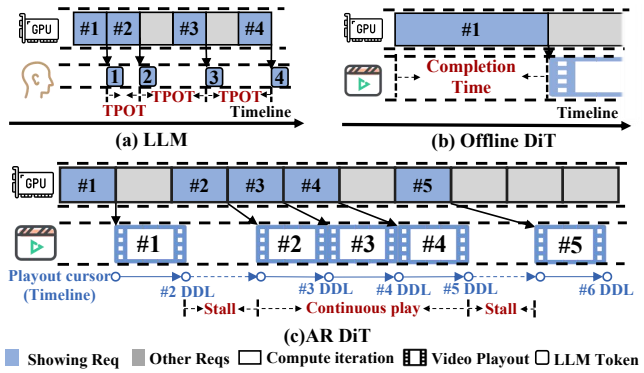

Figure 2. Representative SLOs across LLM, offline DiT, and AR-DiT serving. #i denotes the *i*-th compute iteration, which produces one token in LLMs and one chunk in AR-DiTs.

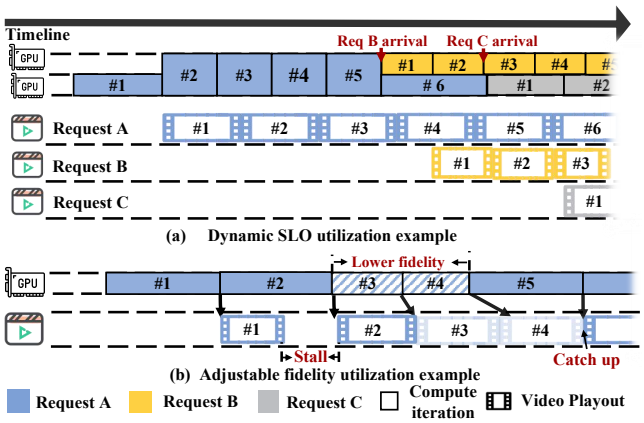

Figure 3. Examples that utilize the properties of AR-DiT.

video stream[1]'s SLO threshold changes after admission. Its SLO evolves as playout advances: the playout cursor moves at fps frames per second, continuously updating the effective deadline (DDL) of the next chunk. This makes resource allocation decisions and SLO evolution mutually dependent. In other words, resource allocation changes a stream's playout slack, while playout slack in turn guides how compute should be allocated[2]. For example, as shown in Figure 3(a), when the system load is light (only stream A is active), it can allocate additional compute resources, such as increasing parallelism, to accumulate playout slack. When load increases (streams B and C arrive) and compute becomes scarce, stream A remains non-urgent due to accumulated slack, allowing resources to be allocated to streams B and C as they are under heavier playout pressure. Such dynamic SLO control cannot be captured by a fixed metric like TPOT.

*Property 2: Real-time video generation has **adjustable fidelity** due to a deadline-sensitive quality-latency tradeoff: timely chunks can outweigh late high-fidelity chunks. Playout continuity* can be tracked by an objective runtime quantity: playout slack. When playout slack is exhausted, *playout continuity* is violated and users immediately perceive stalls. Thus, when system load is heavy, it is acceptable to temporarily deliver a chunk with lower fidelity on time, rather than a high-fidelity chunk late. The same principle is widely used in deadline-sensitive scenarios, such as adaptive-bitrate streaming over pre-encoded segments (e.g., Pensieve, BBA, and MPC) [19, 40, 74], and dynamic resolution scaling for game rendering [30]. Fortunately, the chunk-wise generation pattern of AR-DiTs allows the serving system to select a fidelity configuration for each chunk based on current playout slack. As exemplified in Figure 3(b), the system can temporarily choose lower-cost fidelity configurations when playout slack is running out, rather than fixing stream-wide quality.

These properties open a design space largely unexplored by existing serving paradigms: resources must be reallocated across streams, and fidelity must be adjusted across chunks according to the real-time distribution of playout slack (§2.3).

While these properties open a new design space, realizing them in a stable serving system raises two challenges.

Challenge 1. Playout slack dynamics make resource allocation decisions difficult. At the algorithm level, resource allocation and slack evolution continuously affect each other (§4). Therefore, making ad-hoc resource allocation decisions at runtime is challenging in this interdependent decision space. At the system level, AR-DiTs are stateful, typically through KV cache [20, 61], so reallocating compute often requires state migration and introduces overhead.

Challenge 2. Multidimensional fidelity knobs complicate the speed–quality tradeoff. Unlike adaptive-bitrate streaming [19], which mainly changes transmission bitrate, AR-DiTs expose multiple speed–quality tradeoff knobs, such as sparse attention and quantization [25, 33, 39, 72, 78], whose combinations create a complex decision space (§2.1, §5). As a result, fidelity-configuration selection depends on complex interactions among workload state, serving pressure, and quality requirements, making the tradeoff non-trivial (§5).

To address these challenges, we present ***SlackServe***, a playout-slack-driven serving system for real-time streaming video generation. To our knowledge, SlackServe is the first system built around *dynamic SLOs* and *adjustable fidelity* to preserve *playout continuity* in real-time video generation.

To handle dynamic SLOs, SlackServe introduces *Slack-Driven Resource Reallocation* (§4), which combines two common resource-reallocation mechanisms: preemption and compute expansion. Specifically, SlackServe uses three-tier priority queues (§4.1) and cross-worker re-homing (§4.2) for local and remote preemption, respectively, and uses elastic sequence parallelism [23, 34, 66] (elastic SP) (§4.3) for compute expansion on urgent streams. SlackServe jointly coordinates these mechanisms with adjustable fidelity, composing and

[1]A video stream is a real-time video generation session.
[2]Playout slack can also change due to user behavior, including prompt switches that reset it and playout pauses that let it accumulate.

triggering them in appropriate orders to make reallocation decisions in the interdependent decision space.

To enable adjustable fidelity, SlackServe introduces *Slack-Driven Fidelity Selection* (§5). It first establishes a profiled Pareto frontier [16, 41] over the fidelity-configuration space, capturing the non-dominated speed–quality tradeoffs among candidate fidelity configurations. Based on this frontier, SlackServe designs *Bi-Modal Pareto Routing (BMPR)*, which routes chunks to suitable fidelity configurations according to the current playout-slack budget while enforcing a global quality floor through a bi-modal decision rule.

At the system level, SlackServe realizes these mechanisms through three decoupled asynchronous planes (§3.2). The *Control Plane* jointly makes resource allocation and fidelity selection decisions based on playout slack. The *Execution Plane* dispatches streams and applies the *Control Plane*'s decisions. The *State Plane* manages the paged KV cache and performs asynchronous migration according to these decisions. This design decouples decision making, execution, and state migration while overlapping migration with ongoing chunk computation whenever possible.

We summarize our main contributions as follows.

- We present SlackServe, to our knowledge the first serving system designed to preserve playout continuity in real-time video generation by exploiting two properties of streaming generation: dynamic SLOs and adjustable fidelity.
- Building on dynamic SLOs, we design *Slack-Driven Resource Reallocation*, which combines three-tier priority queues, re-homing, and elastic SP to mitigate resource mismatch and direct compute toward streams at risk of stalling.
- Building on adjustable fidelity, we design *Slack-Driven Fidelity Selection*, which uses Bi-Modal Pareto Routing (BMPR) to select a Pareto-optimal fidelity configuration for each chunk under a time budget and quality floor, balancing playout continuity with global quality preservation.
- We evaluate SlackServe on a 16×H100 GPU cluster. At comparable generation quality, SlackServe improves *Quality of Experience* (QoE), measured by *Continuous Play Ratio* (CPR), by 1.64×–3.29× and reduces *Time to First Chunk* (TTFC) by 1.61×–9.65× over baselines.

2 Background and Motivation

Table 1 summarizes the terminology used in the paper.

2.1 Video Generation

As shown in Figure 1(a), video generation [20, 28, 51, 54, 63, 80] typically follows a three-stage latent *diffusion pipeline* [6, 18, 59]. First, an encoder [53] maps conditional inputs, such as prompts and references, into conditioning embeddings. Second, conditioned on these embeddings, a DiT model iteratively denoises latent noise over multiple steps [18, 59] to generate a latent video. Third, a decoder [54] maps the latent video into pixel-space video. Existing models built around this pipeline mainly fall into two paradigms: offline one-shot generation and chunk-wise autoregressive generation.

Offline DiT. As shown in Figure 1(b), offline DiTs [51] treat an entire video as a single diffusion process: all latent frames are denoised together to produce the whole video. Because all frames are processed jointly, attention spans the full video, causing attention cost to grow quadratically with the number of frames [62]. It therefore does not scale well to long videos or low-latency real-time settings [67, 69].

AR-DiT. AR-DiTs [20, 36, 61, 80] reformulate video generation as a chunk-by-chunk process. As shown in Figure 1(c), the model generates one short video segment at a time, which we call a *chunk*. The chunk becomes immediately available for playout once generated and is then appended to a rolling KV cache for subsequent chunks.

To keep the KV cache bounded for long streams, AR-DiTs commonly adopt a *sink + local* strategy [20, 31, 70], which preserves the first few chunks as a global sink and keeps KV entries only for the most recent W chunks. This allows a stream to extend, in principle, indefinitely under bounded GPU memory. CausVid [73], Self-Forcing [20], and Rolling Forcing [36] follow this strategy, making AR-DiTs a key foundation for real-time streaming video generation.

Fidelity knobs and configurations. Real-time video generation exposes several serving-time knobs that trade generation latency for visual fidelity. We call each such dimension a *fidelity knob*, and a concrete assignment of these knobs a *fidelity configuration*. In this paper, a fidelity configuration includes denoising steps S, attention sparsity ρ, KV-window size W, and quantization mode Q. (1) *Denoising Steps.* This knob controls the number of denoising steps S in DiT models. Reducing S roughly lowers latency linearly, but can alter the speed–quality tradeoff in diffusion deployment [13, 37, 59]. (2) *Sparse Attention.* It skips non-selected attention blocks with sparse masks and is controlled by attention sparsity ρ [11, 67, 69]. (3) *KV Window Size.* It controls the maximum KV-cache window size W used by attention, following sliding-window and attention-sink mechanisms in streaming autoregressive generation [31, 36, 70]. (4) *Quantization.* It compresses weights or activations into a lower-precision mode Q, such as FP8, exploiting tensor cores to accelerate compute and memory access [56, 78]. These knobs define the fidelity-configuration space for serving-time speed–quality tradeoffs in real-time video generation. Importantly, for a fixed target resolution, chunk size, and fidelity configuration, DiT chunk latency is highly profileable offline [2, 44, 68], providing the timing prior used by SlackServe for resource allocation (§4) and fidelity selection (§5).

2.2 Resource Allocation in Serving Systems

Serving systems allocate compute resources along two complementary dimensions.

The first is temporal allocation, which determines which request should receive service earlier. Its central mechanism is preemption, where execution is reordered at safe boundaries to favor more urgent requests. Preemption can occur within a request or across requests. Within a request, techniques such as continuous batching [75] expose fine-grained execution boundaries and allow the system to interleave work across iterations [1, 29]. Across requests, priority scheduling preempts less urgent requests and lets latency-sensitive ones run earlier [79]. Temporal allocation changes when a request receives service, but not the amount of compute assigned to it at a given time.

The second dimension is spatial allocation, which changes a request's compute share by scaling execution across devices. Its central mechanism is parallelism, including tensor [58], pipeline [43], and sequence parallelism [23, 34, 66, 68]. For DiTs, sequence parallelism (SP) is especially relevant because video generation creates long token sequences. SP partitions the sequence across GPUs, lets each GPU process one shard, and uses collective communication to exchange states required by attention. Increasing the sequence-parallel degree can reduce per-chunk latency, but it also occupies more workers and may require state redistribution.

2.3 Motivation

Limitations of existing serving systems. For scheduling and resource allocation, existing LLM serving systems such as vLLM [29], Sarathi-Serve [1], DistServe [79], and LoongServe [66] are largely designed around request-level latency and throughput objectives, typically expressed as *Time to First Token* (TTFT), *Time per Output Token* (TPOT), or goodput [79]. Text streaming serving systems, such as Andes [35], TokenFlow [10], have explored satisfying user quality of experience (QoE) by introducing preemptive request scheduling methods. Although these systems draw inspiration from QoE in video streaming [4], the technical challenges are largely different and cannot be directly applied to real-time video generation serving (§8). Systems for diffusion and video serving, such as DiffServe [2], TridentServe [68], and TetriServe [38], mainly optimize throughput or completion time for offline generation workloads [68]. More recently, StreamDiffusionV2 [14] targets streaming video generation through batching and pipeline parallelism [21, 43], but still optimizes frames-per-second (*FPS*) objectives rather than explicitly tracking each stream's real-time playout slack. For acceleration knobs, methods such as AWQ [33], SparseVideoGen [67], and SageAttention [78] improve speed through static quality-affecting configurations, which are chosen before serving and remain unchanged at runtime.

Table 1. Terminology used in the problem formulation.

Term	Meaning
Stream	One real-time video generation session.
Frame	A single image in the generated video stream.
Chunk	A short video segment containing multiple consecutive frames, generated and delivered as one unit in streaming video generation.
Playout continuity	Chunks are ready before playback reaches them, avoiding visible stalls.
Playout slack	Playable video buffer already generated but not yet consumed by playback.

However, in real-time generation, the goal is not merely faster computation but preserving *playout continuity*, which is critical to user experience. Existing serving systems do not explicitly exploit this objective.

Motivating Insights. With *playout continuity* as the primary goal, we derive two system insights.

Insight 1. Compute resources should be reallocated according to playout slack. Because playout slack evolves over time, preserving *playout continuity* requires resource allocation to track this evolution and redirect compute toward streams under tighter playout pressure. Existing serving systems do not make allocation decisions based on this per-stream slack signal, so urgent streams can remain under-served and eventually stall. Our case study with StreamDiffusionV2 (§7.4) shows that even when average system FPS matches the playout rate, some streams still stall due to the mismatch between playout slack and resource allocation.

Insight 2. Each chunk's fidelity configuration should adapt to the current playout slack. Since users are highly sensitive to stalls, delivering an acceptable chunk on time is often more valuable than a late high-fidelity chunk. This principle is widely adopted in streaming media [19, 40, 74]. Offline video generation fixes one fidelity configuration for the whole video due to one-shot execution. In contrast, AR-DiTs generate one chunk at a time, allowing the system to reduce latency when slack is tight and restore fidelity when slack recovers (§7.2, §7.4).

These insights define the design space of SlackServe: Driven by playout slack, it reallocates resources across streams toward urgent streams (§4) and selects per-chunk fidelity configurations within streams to balance *playout continuity* and visual quality (§5).

3 System Overview

Based on these insights, we develop SlackServe, to our knowledge, the first playout-slack-driven serving system for real-time streaming video generation.

Below, we first introduce the core runtime concepts of SlackServe (§3.1), then its system components (§3.2), and finally its complete workflow (§3.3).

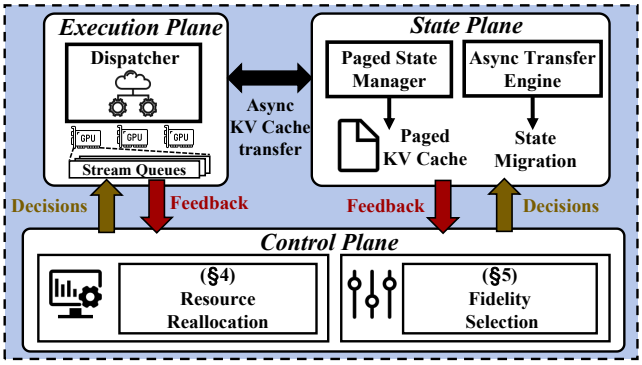

Figure 4. System overview and components of SlackServe.

3.1 Core Runtime Concepts

Service Credit. *Service credit* is derived from *playout slack* to quantify stream urgency. Both resource-allocation and fidelity-selection decisions use *service credit* as the primary control signal. For each active stream u, we define its *service credit* C_u as:

$$C_u = P_u - (R_u + T_u) \tag{1}$$

where P_u is the playout slack (i.e., the remaining playable buffer), R_u is the estimated remaining time to finish the current chunk (zero if the stream is not running), and T_u is the profiled generation time of the next chunk under the fidelity configuration selected for the next chunk (§5). Thus, service credit converts the dynamic playout SLO into a per-stream urgency score.

Step/Chunk Boundaries. As shown in Figure 1, a step boundary occurs after one DiT denoising step, while a chunk boundary occurs after one chunk completes. These boundaries are safe points for resource allocation and fidelity-configuration selection.

Home Worker. Each stream is assigned a *home worker*[3] at admission. By default, a stream's generation and KV cache remain on its *home worker*.

3.2 System Components

Figure 4 shows the system architecture. SlackServe is organized into three planes:

Control Plane. The *Control Plane* is a global controller that wakes up at each *control tick* (every 3 s by default) to make resource-allocation and fidelity-selection decisions. It integrates *Slack-Driven Resource Reallocation* (§4) and *Slack-Driven Fidelity Selection* (§5) into a single control loop. It tracks each stream's service credit C_u, decides which streams should run at the next step or chunk boundary, and makes the corresponding resource-allocation and fidelity-configuration decisions to better preserve *playout continuity*.

State Plane. The *State Plane* (§4.4) uses a *Paged State Manager* to manage KV cache and page GPU-resident KV at latent-frame granularity[4]. To support resource reallocation, it also includes an *Async Transfer Engine* that asynchronously migrates KV state.

Execution Plane. The *Execution Plane* maintains stream queues per worker. At each step or chunk boundary, the *Execution Plane* dispatches the next stream according to decisions from the *Control Plane*.

The three planes interact through lightweight interfaces, where the *Control Plane* makes *decisions* to guide the *Execution* and *State Planes*, while the *Execution Plane* and *State Plane* provide feedback for future decisions.

3.3 System Workflow

We now describe the workflow enabled by the three-plane design in Figure 5. For each stream, the workflow consists of two phases, *stream admission* and *steady scheduling*.

Stream admission. When a new request arrives, ① SlackServe assigns an initial playout slack as its *Time to First Chunk* (TTFC) constraint, set to 4× the estimated first-chunk generation time by default. ② SlackServe then selects the worker with the fewest streams as the stream's *home worker*, avoiding already congested workers. After admission, the stream enters *steady scheduling*.

Steady scheduling. In steady scheduling, the system is driven by two types of events, periodic control ticks and streams reaching step or chunk boundaries.

At each control tick, the *Control Plane* makes execution and migration decisions for the next step or chunk boundary. ③ First, based on current playout slack, it performs fidelity selection (§5) and chooses each stream's next-chunk fidelity configuration. ④ Second, it performs resource reallocation (§4) and decides which streams should run and where they should run. Given the selected fidelity configurations, it recomputes stream service credits to determine which streams should run at the next step or chunk boundary (§4.1) and on which worker(s) they should run (§4.2 and §4.3). These joint decisions shift resources across streams toward urgent streams to better preserve *playout continuity*. Third, if a decision requires state migration, ⑤ the *State Plane* (§4.4) starts asynchronous KV transfer and overlaps it with computation when possible.

At each step or chunk boundary, ⑥ the *Execution Plane* dispatches and executes streams using the assigned workers and fidelity configurations according to the *Control Plane*'s decisions. ⑦ After each step, streams are requeued for the next dispatch decision. Unless the *Control Plane* updates its decision at a later control tick, streams continue under the current decision.

[3]We use *worker* uniformly for the unit of scheduling and execution. By default, one worker corresponds to one GPU and one model replica.

[4]Since different streams may have different KV lengths, frame-level paging reduces fragmentation.

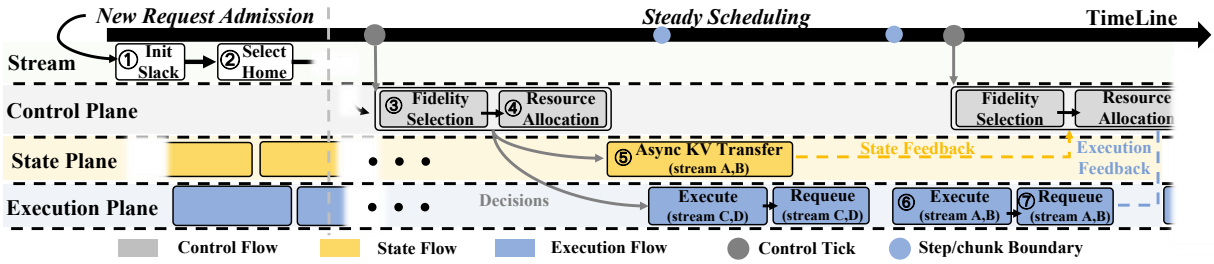

Figure 5. System workflow of SlackServe.

Table 2. Resource reallocation mechanisms, organized by preemption and compute expansion.

	Expansion (✗)	Expansion (✓)
Preemption (✗)	–	Elastic SP (§4.3)
Preemption (✓)	Three-tier queue (§4.1), re-homing (§4.2)	Three-tier queue, re-homing, elastic SP (§4.1–4.3)

In §4 and §5, we detail how the *Control Plane* guides resource reallocation and fidelity selection, respectively. Appendix C.1 summarizes the complete control loop.

4 Slack-Driven Resource Reallocation

This section details how SlackServe reallocates compute resources in real-time video generation.

In a nutshell, our goal is to redirect compute resources from streams with sufficient playout slack to streams at risk of stalling. To achieve this goal, there are two general strategies. The first is preemptive scheduling, which prioritizes urgent streams and defers non-urgent ones. The second is per-stream compute expansion, which increases parallelism by borrowing compute resources for an urgent stream so it can finish before its playout slack is exhausted.

As shown in Table 2, SlackServe realizes these strategies with three mechanisms: per-worker three-tier priority queuing for preemption on each worker (§4.1), cross-worker re-homing for redistributing streams across workers (§4.2), and parallelism scaling for temporary compute expansion (§4.3). Finally, §4.4 presents the State Plane that supports state migration required by these mechanisms.

Guided by service credit, the *Control Plane* composes these mechanisms into a staged resource-allocation loop. First, it orders streams in each worker's queue, enabling local preemption at the next boundary. Then, it detects cluster-wide imbalance and generates a re-homing plan for cross-worker preemption. Finally, for streams still prone to violate *playout continuity*, it expands computing through parallelism scaling. These approaches can be combined under severe slack pressure.

4.1 Three-Tier Priority Queue and Preemption

Three-tier priority queues provide local preemption and the urgency signal used by later reallocation mechanisms.

At each control tick, the *Control Plane* computes each stream's service credit C_u and classifies it as *URGENT* ($C_u < 2T_u$), *NORMAL* ($2T_u \le C_u \le 4T_u$), or *RELAXED* ($C_u > 4T_u$). It then orders streams within each worker by service credit, so lower-credit streams are dispatched earlier at step or chunk boundaries. This local preemption prevents slack-deficient streams from waiting behind relaxed ones and supplies the tier information used by re-homing and elastic SP.

Credit-aware Eviction. Because preempted streams may leave their KV cache in a worker's limited GPU KV pool, the most urgent stream may find its state non-resident when it is rescheduled. When the pool is full, SlackServe evicts the highest-credit resident stream, i.e., the stream least likely to stall, to make room for the urgent stream's state (Figure 8).

4.2 Re-homing

Each stream is bound to a *home worker* at admission and does not migrate by default, because it carries state such as KV cache, and frequent migration incurs substantial communication overhead. In practice, however, we often observe an imbalance between URGENT-heavy workers and RELAXED-only workers (§7.4). We refer to them as *URGENT workers* and *RELAXED workers*, respectively. Migrating URGENT streams from *URGENT workers* to *RELAXED workers* can relieve local congestion and improve workload balance. Figure 6 shows a typical case, where W0 and W1 have only RELAXED streams while W2 has several queued URGENT streams. Without migration, low-credit streams on W2 would repeatedly stall. SlackServe re-homes URGENT streams to W0 and W1, redirecting RELAXED-worker capacity toward urgent streams.

Bipartite Re-homing Planning. We generate migrations using *Bipartite Re-homing Planning*, which performs capacity-bounded, intra-node-preferred matching.

At each control tick, SlackServe matches overloaded workers to slack-rich workers. It treats workers with congested URGENT queues as senders and workers with no URGENT or NORMAL streams as receivers, so migration moves urgent work only toward spare slack capacity (lines 1–2). Receivers are tried in intra-node-first order to reduce transfer cost (line 5). Subject to send/receive caps, SlackServe moves the

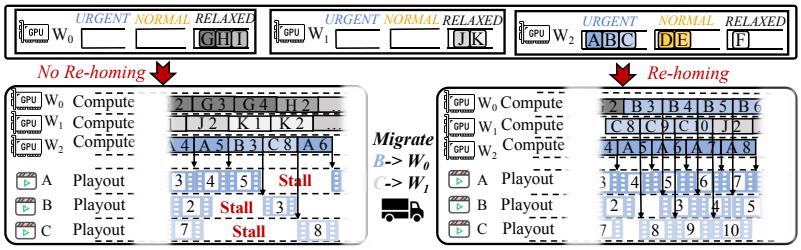

Figure 6. Comparison with and without re-homing.

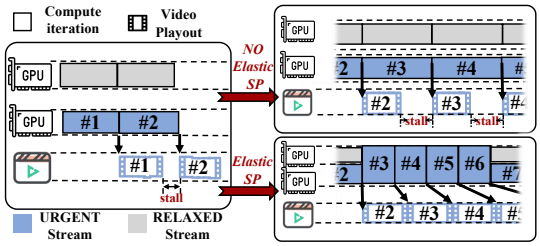

Figure 7. Comparison with and without elastic SP.

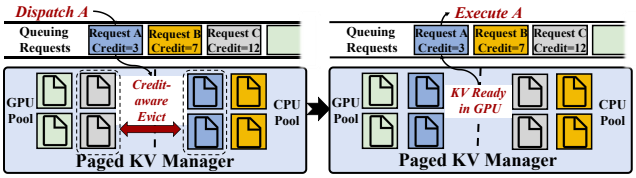

Figure 8. Credit-aware eviction example.

Algorithm 1 Bipartite Re-homing Planning

Input: worker queues with urgency tiers and service credits
Output: migration plan *plan*

1: *senders* ← URGENT-heavy workers
2: *receivers* ← workers with no URGENT or NORMAL streams
3: *plan* ← ∅
4: **for** *src* ∈ *senders* **do**
5: **for** *dst* ∈ *receivers* in intra-node-first order **do**
6: **if** $src.sent < cap_{\text{send}}$ and $dst.recv < cap_{\text{recv}}$ **then**
7: s ← lowest-credit URGENT stream on *src* not in cooldown
8: append (s, src, dst) to *plan*; update caps; mark s in cooldown
9: **return** *plan*

lowest-credit URGENT stream not in cooldown to an available receiver at its next chunk boundary (lines 6–8). This bounded plan improves balance while limiting migration bursts and repeated movement. Appendix C.2 gives the concrete details.

4.3 Elastic Sequence Parallelism

Priority scheduling and re-homing may still fail to recover streams with extremely low service credit, due to estimation errors or runtime fluctuations. As shown in Figure 7, SlackServe uses elastic SP as a last-resort recovery mechanism when a stream has $C_u < 0$, meaning it is projected to miss its next playout window. In this case, SlackServe borrows the highest-credit RELAXED worker as a donor and switches the stream to the corresponding pre-initialized intra-node SP2 group, accelerating the current chunk through sequence parallelism. The donor is released at the next safe boundary once the stream recovers to the NORMAL tier ($C_u \geq 2T_u$).

By default, SlackServe restricts elastic SP to intra-node SP2 and borrows at most one donor worker. All candidate SP2 groups are pre-initialized before serving, so elastic SP only switches the stream's active execution group rather than creating communication groups on the critical path. The required head-partition KV transfer is handled asynchronously by the State Plane (§4.4); Appendix C.3 and Appendix C.4 give the full policy and transfer details.

4.4 State Plane

Credit-aware eviction (§4.1), re-homing (§4.2), and elastic SP (§4.3) are functionally distinct but share the same KV-state transfer primitive. SlackServe therefore centralizes state movement in an independent *State Plane*, with a single transfer interface, shared non-blocking layer-wise streaming, and atomic-safety guarantees.

Unified KV management. As shown in Figure 9, each worker maintains a paged KV pool, defaulting to $\kappa = 0.8$ of available VRAM [29]. Pages are allocated at frame granularity (one page per latent frame) and mapped to physical pages through a logical page table, avoiding fragmentation.

The three state-transfer operations share one interface:

```
transfer(stream, src, dst, page_range)
```

where each caller specifies the required `page_range`. This lets mechanisms in §4.1–§4.3 specify only which pages should reside where, leaving transfer timing to the *State Plane*.

Asynchronous streaming transfer. Synchronous KV movement can block dispatch and erase the benefit of reallocation. In SlackServe, transfer requests return immediately and are executed by an *Async Transfer Engine* on dedicated CUDA streams. The engine uses one protocol for eviction, re-homing, and elastic SP (Figure 9):

- *Non-blocking.* Page transfers are scheduled asynchronously subject to dependency and bandwidth limits, so foreground dispatch and chunk computation do not wait for a full transfer to complete.
- *Atomic safety.* The dispatcher must never schedule a stream with an incomplete state. Once a transfer is submitted, the target stream is temporarily removed from

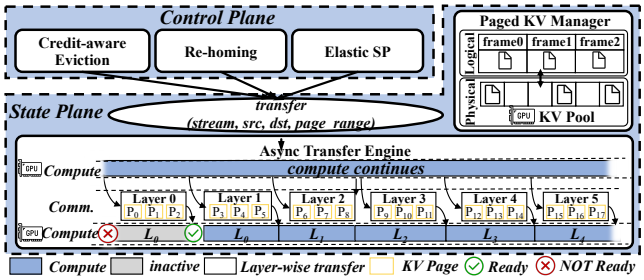

Figure 9. State Plane design with unified KV management and asynchronous streaming transfer.

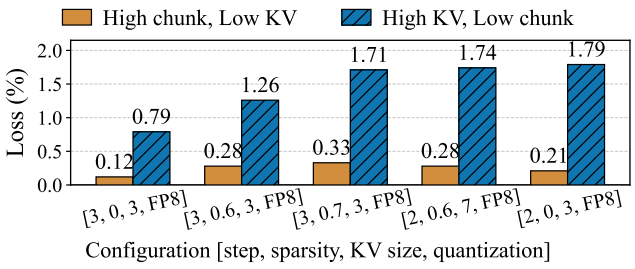

Figure 10. Quality impact of low-precision current chunks and low-precision historical KV cache.

the active queue. The stream is reinserted once its first-layer state is ready, enabling layer-wise computation transfer overlap while preserving correctness.

- *Layer-wise streaming*. Transfers are issued in layer-major order. Once layer k's pages arrive, the destination can start layer-k computation while later layers continue transferring, overlapping computation with state movement.

5 Slack-Driven Fidelity Selection

This section introduces how SlackServe makes per-chunk fidelity-configuration decisions, applied at chunk boundaries, based on playout slack. We construct a fidelity-configuration space from four knobs: denoising steps S, attention sparsity ρ, KV-window size W, and quantization mode Q (detailed in Appendix A). A concrete assignment forms a fidelity configuration $\text{cfg}_i = \{S_i, \rho_i, W_i, Q_i\}$. These knobs control the speed–quality tradeoff[5].

Making fidelity-selection decisions is challenging. On the one hand, the complex multidimensional search space makes online fidelity-configuration selection difficult under a playout-slack budget. On the other hand, choosing only the fastest fidelity configuration within the playout-slack budget can push the system toward low-quality fidelity configurations under overload, causing visible degradation.

To address this, we propose *Bi-Modal Pareto Routing (BMPR)*. BMPR uses an empirical latency–quality Pareto frontier and a global quality floor to make a bi-modal decision: it prioritizes quality when playout slack permits, and enforces the quality floor when playout slack is tight.

We first present a key observation (§5.1) that motivates our design. We then describe BMPR's offline construction and online selection procedure (§5.2).

[5]We use VBench [22], a widely used video-generation benchmark, to measure generation quality.

5.1 Observation: quality loss has limited propagation through KV cache

In Figure 10, we test whether a low-precision configuration used for previous chunks degrades the current chunk through the rolling KV cache.

We isolate the source of quality loss by comparing two settings against the all-high-quality reference. In the first setting, the current chunk uses the highest-quality fidelity configuration, while its KV cache is produced by a low-cost fidelity configuration cfg_i. In the second setting, the current chunk uses cfg_i, while its KV cache is produced by the highest-quality fidelity configuration. We measure the percentage drop in VBench [22] for the current chunk. The results show that low-fidelity historical KV causes only a small quality drop, whereas lowering the current chunk's fidelity configuration causes a much larger drop.

This observation shows that quality degradation has limited propagation through KV cache, making per-chunk fidelity-configuration decisions largely independent: downshifting an urgent chunk does not imply persistent quality degradation in later chunks.

5.2 Bi-Modal Pareto Routing

Based on this observation, we present Bi-Modal Pareto Routing (BMPR), which uses an offline empirical latency–quality Pareto frontier with a global quality floor Q_{floor} for online fidelity selection.

Offline construction. First, SlackServe profiles each candidate's fidelity configuration and represents it as a point (L, Q) in the latency–quality plane, with details given in Appendix A. A fidelity configuration i is dominated if another fidelity configuration j satisfies $L_j \le L_i$ and $Q_j \ge Q_i$, with at least one strict inequality. The non-dominated fidelity configurations form the empirical Pareto frontier:

$$F = \{(L_i, Q_i, \text{cfg}_i) \mid \text{cfg}_i \text{ is non-dominated}\}.$$

This frontier removes fidelity configurations for which another fidelity configuration is no slower and no lower-quality, so online selection considers only meaningful speed–quality tradeoff points.

Second, to avoid visible degradation from aggressive downshifting, SlackServe uses the median quality over all candidate fidelity configurations as the global quality floor Q_{floor}.

Online selection. Given a playout-slack budget B for the current chunk, BMPR selects a fidelity configuration from F in two modes:

- *Quality mode.* By default, BMPR filters frontier points with $L_i \leq B$ and $Q_i \geq Q_{\text{floor}}$. If the set is non-empty, BMPR selects the highest-quality fidelity configuration. This mode targets normal or mildly congested conditions, preserving quality while staying within the playout-slack budget.
- *Speed-recovery mode.* Quality mode is infeasible when no configuration within the budget also satisfies the quality floor. In this case, BMPR avoids blindly choosing the fastest point regardless of quality. Instead, it selects the min-latency point among fidelity configurations with $Q_i \geq Q_{\text{floor}}$. This choice may exceed the current budget but prevents visible quality loss.

The speed-recovery mode may still violate *playout continuity*. In that case, the resource reallocation mechanisms in §4 provide the next line of defense. This joint design of bi-modal fidelity selection and resource allocation preserves *playout continuity* while bounding visual degradation. §7.5 shows that BMPR maintains stable quality even when higher arrival rates create more low-credit chunks, rather than degrading linearly with congestion.

6 Implementation

SlackServe is implemented in a custom serving framework of about 18K lines of Python code and supports representative AR-DiT models, including CausVid [73], Self-Forcing [20], Causal-Forcing [80], and Rolling Forcing [36]. We use Ray to manage cluster workers [42]. The *Control Plane* runs on an `asyncio` event loop [52] and coordinates through Ray actors [42]. SlackServe also includes an offline profiler that provides latency and quality profiles for fidelity selection and service-credit estimation.

For fidelity configurations, we implement sparse attention based on Light Forcing [39] and use SageAttention2 [77] for online FP8 attention quantization. SageAttention2 dynamically quantizes and dequantizes activations online without quantizing model weights, so switching fidelity configurations does not require weight reloading [77].

The unified transfer interface in §4.4 is implemented by a NIXL-based transfer engine [3]. It uses NCCL P2P over NVLink within a node [48, 50] and the NIXL RDMA backend across nodes [3, 46]. All transfers run on independent CUDA streams and synchronize with compute streams through CUDA events [45].

7 Evaluation

7.1 Experimental setup

Testbed. We evaluate SlackServe on a two-node cluster with 16 NVIDIA H100 80GB GPUs. GPUs within a node are connected by NVLink with up to 900 GB/s per GPU [47, 48], and the two nodes are connected by 400 Gb/s InfiniBand [49].

Models. We use two representative AR-DiT models, Causal-Forcing [80] and Self-Forcing [20]. By default, each chunk contains three latent frames, with 480p resolution and a 16 fps playout rate.

Workloads. We construct five workloads: four synthetic ones and one proprietary enterprise trace for evaluation.

- **Steady**: streams arrive according to a Poisson process [55] at 1 stream/s.
- **Burst**: extends Steady with three burst points, each causing 10% of all streams to arrive simultaneously.
- **Prompt-switch**: extends Steady by injecting condition switches at random positions in every stream, resetting playout slack.
- **Pause**: extends Steady by adding client-side pauses to every stream, during which playout stops and slack accumulates.
- **Proprietary trace**: a scaled enterprise trace with mixed steady, bursty, and idle arrival patterns.

We use all 946 prompts from VBench [22] and sample each stream length from $\{81, 129, 161, 241\}$ frames, corresponding to about 5–15 s. Appendix B details all workloads.

Baselines. We compare SlackServe against three representative external baselines.

- **StreamDiffusionV2 (SDV2)** [14] represents streaming-video serving that optimizes frame-rate objectives with FIFO and batching.
- **TridentServe (TS)** [68] represents DiT serving with dynamic resource management, including dynamic parallelism and migration, but manages SLOs at the per-stream level.
- **TridentServe-chunk (TS-chunk)** is a slack-aware baseline which extends TS to per-chunk SLO management: it uses least-slack-first scheduling with static fidelity, while retaining TS's dynamic parallelism and re-homing mechanisms.

Because no existing open-source system fully supports playout-slack-driven AR-DiT serving, §7.3 uses an ablation study to evaluate stronger controlled variants of SlackServe that cover the main competing design points.

Metrics. We use three metrics that directly reflect the real-time video experience.

- **Quality of Experience (QoE)**: We define QoE as user-side playout continuity, measured by *Continuous Play Ratio* (CPR), the fraction of chunks delivered before

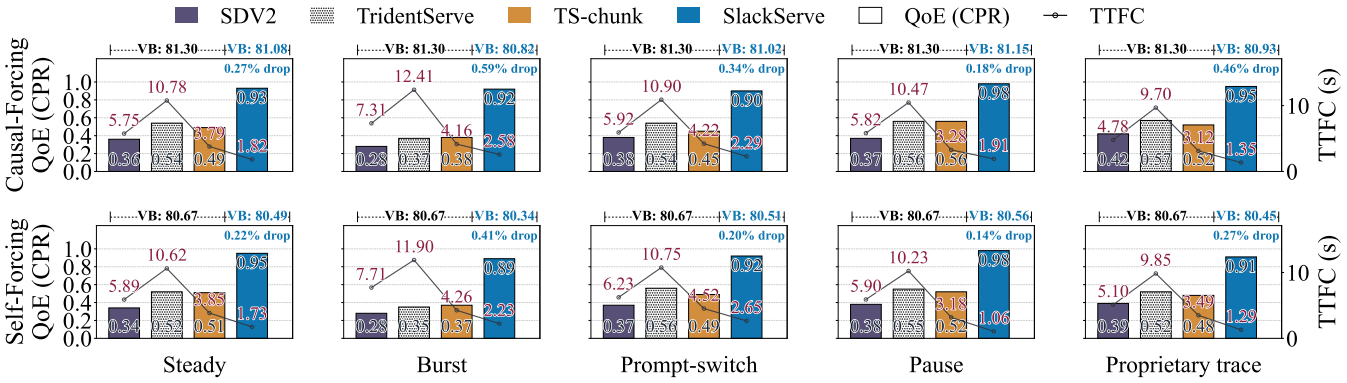

Figure 11. End-to-end comparison on Causal-Forcing and Self-Forcing across the five workloads.

their playout deadlines:

$$\text{QoE} \equiv \text{CPR} = \frac{1}{|\mathcal{U}|} \sum_{u \in \mathcal{U}} \frac{1}{N_u} \sum_{i=1}^{N_u} \mathbf{1}\{t_{u,i}^{\text{ready}} \le t_{u,i}^{\text{ddl}}\}.$$

Here $t_{u,i}^{\text{ready}}$ is the ready time of chunk i in stream u, and $t_{u,i}^{\text{ddl}}$ is its playout deadline. Higher QoE indicates fewer playout-continuity violations.

- **Time to First Chunk (TTFC)**: the time from stream arrival to delivery of the first playable chunk to the client, analogous to time-to-first-output metrics in generative serving [79].
- **VBench** [22]: a widely used composite benchmark for video generation, covering visual quality, temporal consistency, motion quality, and semantic alignment.

7.2 End-to-end results

We compare SlackServe with StreamDiffusionV2 (SDV2), TridentServe (TS), and TridentServe-chunk (TS-chunk) across both models and all five workloads in Figure 11.

SlackServe consistently improves QoE and reduces TTFC while maintaining comparable generation quality. Across the five workloads and two models, SlackServe improves QoE by 2.65×, 1.88×, and 1.98× on average over SDV2, TS, and TS-chunk, respectively, and reduces TTFC by 3.39×, 6.10×, and 2.11×. Across individual settings, these improvements span 1.64×–3.29× for QoE and 1.61×–9.65× for TTFC. The VBench drop is below 0.6% across all settings, indicating comparable benchmark-level generation quality [22, 67, 69].

The workload breakdown shows that SlackServe is effective under different pressure patterns. Under Steady and Burst, SlackServe keeps QoE high despite continuous load and synchronized arrivals. Under Prompt-switch, where condition changes reset accumulated slack, SlackServe maintains QoE of 0.90 on Causal-Forcing and 0.92 on Self-Forcing, compared with 0.37–0.56 for the baselines. Pause is less adversarial because client-side pauses accumulate slack, yet SlackServe again achieves the highest QoE, 0.98 on both models. On the proprietary trace, SlackServe sustains QoE of 0.95 on Causal-Forcing and 0.91 on Self-Forcing, showing that the same control policy remains effective under realistic non-stationary arrivals.

The baselines fall short for different reasons. SDV2 uses batching and pipeline parallelism to improve aggregate FPS, but this increases per-chunk latency and does not prioritize streams whose playout slack is being consumed. TS uses dynamic parallelism to accelerate requests and improves QoE over SDV2, but its control loop operates at the per-stream SLO level and cannot adapt individual chunks to the current slack budget. Its parallelism reconfiguration also delays the first chunk, inflating TTFC. TS-chunk is a slack-aware baseline that uses chunk-level SLOs to guide dynamic parallelism and scheduling, but its policy is designed for offline DiT serving. When applied to AR-DiT streaming, it frequently changes SP degree and migrates KV cache, introducing reconfiguration overhead that is unnecessary for many chunks.

7.3 Ablation study

We run ablations with Causal-Forcing on the Steady workload, varying one design dimension at a time.

Technique ablation. We enable mechanisms in the order in which the *Control Plane* invokes them (Figure 12). In this ablation, *Credit Only* corresponds to least-slack-first scheduling with static fidelity and without dynamic resource reallocation. Credit scheduling alone reaches 0.59 QoE, because boundary preemption gives low-credit streams earlier execution opportunities. Adding BMPR raises QoE to 0.81 by shortening urgent chunks, with only a 0.44% quality drop. Re-homing provides the largest resource-side gain, improving QoE to 0.88 by moving URGENT streams to RELAXED workers. Elastic SP adds the final 0.05, reaching 0.93, by accelerating streams projected to stall even after local scheduling and re-homing. Quality changes little across the resource mechanisms, confirming that they primarily change *where* resources are allocated rather than the fidelity configuration used by each chunk. TTFC first drops from 3.62s to 1.59s after BMPR, then slightly increases to 1.82s as re-homing and elastic SP add migration and coordination overhead.

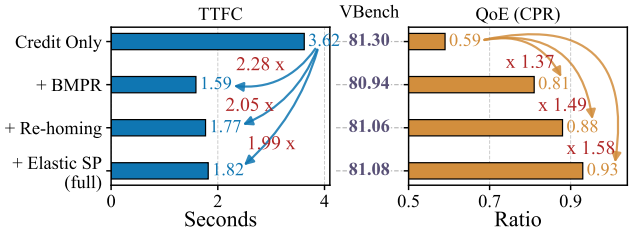

Figure 12. Technique ablation with mechanisms enabled incrementally in Control-Plane trigger order.

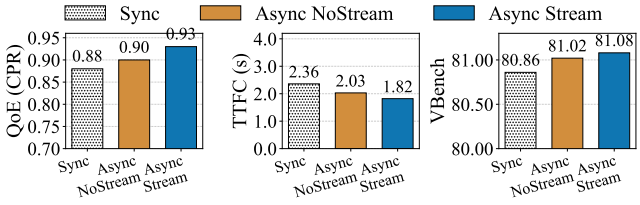

Figure 13. Comparison of State-Plane transfer protocols.

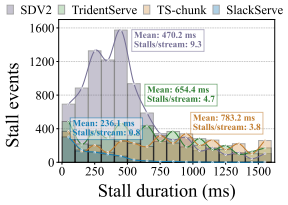

Figure 14. Stall-event duration distribution on the Steady workload.

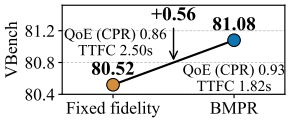

Figure 16. BMPR versus fixed-level switching.

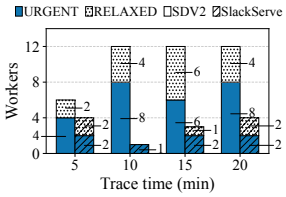

Figure 15. Worker-type distributions for SlackServe and SDV2.

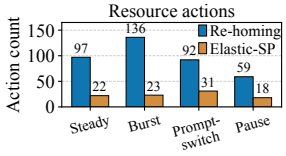

Figure 17. Trigger counts of re-homing and elastic SP.

Comparison with fixed-level fidelity switching. We compare BMPR with a fixed-level switching policy that uses three Pareto-frontier configurations—fast, medium, and slow—and switches among them based on playout slack (Figure 16). Although this policy reaches 0.86 QoE and 80.52 quality, BMPR further improves QoE to 0.93, increases quality by 0.70%, and reduces TTFC from 2.50s to 1.82s. This shows that BMPR's benefit does not come merely from dynamic fidelity switching, but from using the Pareto frontier and quality-floor constraint.

Comparison with different transfer protocols. Re-homing, elastic SP, and eviction all move the KV state. If these transfers block dispatch or computation, they can offset the scheduling benefit (Figure 13). **Sync** blocks the dispatcher until migration, SP splitting, or eviction completes. **Async-NoStream** submits transfers asynchronously but waits for the entire state before target-side computation starts. **Async-Stream**, the default in SlackServe, adds layer-wise streaming and atomic readiness callbacks. Async-Stream reduces TTFC from 2.36s to 1.82s and improves QoE from 0.88 to 0.93. Quality also improves over Sync, since lower blocking overhead leaves more slack for BMPR to stay in quality mode.

7.4 Case study

Stall analysis. Figure 14 shows the stall duration distribution on Steady across systems. SlackServe reduces average stall time from 470–783 ms to 236 ms, and stall frequency from 3.8–9.3 to 0.8 stalls per stream. This confirms that the QoE improvement in Figure 11 corresponds to fewer and shorter visible stalls, not merely a higher deadline-hit ratio.

Imbalance analysis. Figure 15 explains why aggregate FPS alone is insufficient. Although SDV2's average FPS (16.8) reaches the playout target, it leaves 6.5 URGENT and 4.5 RELAXED workers on average, so urgent streams stall while relaxed workers still hold slack. SlackServe keeps URGENT and RELAXED workers near 1.75 and 1.25 through re-homing and local prioritization, which explains the stall reduction.

Resource-allocation decision analysis. Figure 17 shows that SlackServe triggers different decisions under different load patterns. Burst causes the most re-homings (136), as synchronized arrivals create cross-worker slack imbalance. Prompt-switch causes fewer re-homings (92) but more elastic-SP events (31), since condition changes create isolated low-slack chunks that require short-term expansion. Pause needs the fewest decisions (59 re-homings and 18 elastic-SP), because pauses let streams accumulate slack. These results show that service credit steers SlackServe toward re-homing for worker imbalance and elastic SP for tail recovery.

Fidelity-selection analysis. Figure 18 compares the fidelity configurations selected by BMPR under Steady and Burst workloads. The top five configurations account for 94.1% of selections under Steady and 79.4% under Burst, showing that BMPR does not frequently oscillate across the full configuration space. Instead, it repeatedly selects a small set of Pareto-efficient configurations that provide speed–quality tradeoffs. The distribution also shifts with workload pressure. Under Steady load, where playout slack is stable, BMPR selects more high-quality configurations. Under Burst load, synchronized arrivals reduce slack and push more chunks toward faster configurations. This behavior shows that BMPR responds to runtime slack pressure while keeping selections within quality-preserving Pareto choices.

7.5 Sensitivity analysis

We evaluate whether SlackServe depends on tightly tuned parameters. All sweeps use Causal-Forcing on the Steady

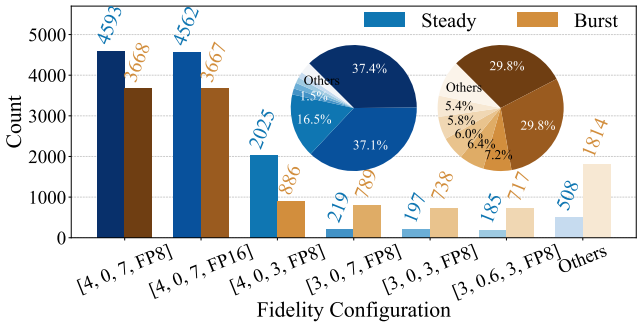

Figure 18. Selected fidelity configs under Steady and Burst.

Table 3. Sensitivity analysis under the Steady workload.

URGENT/RELAXED threshold				Arrival rate			
α	QoE	TTFC	VBench	Rate	QoE	TTFC	VBench
1.0	0.884	1.59	80.82	0.6	0.998	0.81	81.25
1.5	0.925	1.77	81.11	1.0†	0.932	1.82	81.08
2.0†	0.932	1.82	81.08	1.4	0.851	3.92	80.95
3.0	0.928	2.10	80.93	1.8	0.794	6.92	80.67
4.0	0.869	1.76	80.95	2.2	0.733	8.19	80.49

†Default setting. TTFC is in seconds, and Rate is in streams/s.

workload. Parameters outside the sweep use their default values. Results are shown in Table 3.

URGENT/RELAXED threshold. SlackServe classifies a stream as URGENT when $C_u < \alpha T_u$ and RELAXED when $C_u > 2\alpha T_u$. We sweep $\alpha \in \{1, 1.5, 2, 3, 4\}$, and find that QoE is stable for $\alpha \in \{1.5, 3.0\}$: QoE varies only from 0.925 to 0.932 and 0.928, while TTFC remains within 1.77–2.10s. This plateau shows that SlackServe is insensitive to the exact urgency threshold. At the extremes, $\alpha = 1$ marks streams as URGENT too late, causing more elastic-SP recovery and tail stalls, whereas $\alpha = 4$ over-classifies streams as urgent, increasing unnecessary migration. QoE drops to 0.869, with only a 0.16% quality decrease. We therefore use $\alpha = 2$, which lies in the middle of the stable region.

Arrival rate. We sweep the Steady arrival rate from 0.6 to 2.2 streams/s. As load increases, QoE decreases from 0.998 to 0.733, and TTFC rises from 0.81s to 8.19s. The degradation is gradual rather than abrupt, showing that the controller sheds pressure progressively through priority scheduling, re-homing, elastic SP, and BMPR instead of failing at a narrow saturation point. Quality decreases moderately by 0.94% at the heaviest load, indicating that BMPR bounds quality loss even when more chunks enter low-slack states. Even at 2.2 streams/s, SlackServe achieves 0.733 QoE, higher than the Steady QoE of all baselines in Figure 11. This suggests that the gains come from the closed-loop slack-driven design rather than from tuning to the default admission rate.

7.6 Other experiments

We further quantify the scalability of the Control Plane and migration overhead in Appendix D. Overall, these experiments confirm that SlackServe 's *Control Plane* incurs negligible runtime cost, and that asynchronous state migration keeps communication overhead largely off the critical path.

8 Related Work

LLM and DiT serving. LLM serving systems improve throughput and latency through scheduling, KV-cache management, batching, and parallelism [1, 29, 66, 75, 79]. Text-streaming serving systems, such as Andes [35] and TokenFlow [10], further improve perceived user experience through preemptive scheduling and token-level streaming. However, their objectives target token delivery, such as reducing token wait time or improving token-level smoothness, rather than generated-video playout continuity. Unlike text tokens, generated video chunks have explicit playout deadlines and objectively observable stall events. Therefore, these mechanisms do not directly address real-time video generation serving. Diffusion serving systems similarly optimize denoising workloads through scheduling and resource management [2, 32, 38, 68]. Among them, TridentServe [68] dynamically adjusts parallelism to satisfy diffusion-service SLOs, while StreamDiffusionV2 [14] targets interactive video generation through rolling KV cache, batching, and pipeline orchestration. However, these systems optimize request-level latency, throughput, or FPS rather than the continuously evolving playout slack of each video stream. SlackServe instead focuses on preserving playout continuity under streaming generation.

Dynamic fidelity configuration and efficient video generation. A complementary line of work exposes speed–quality tradeoffs for video generation through model-level and kernel-level acceleration. Common techniques include directly reducing denoising steps, with distillation or consistency models further improving quality at low step counts [64, 72], sparse attention [39, 67, 69], KV-window reduction and autoregressive generation [31, 36], and low-precision quantization [77, 78]. Other approaches, such as caching [24, 60], pruning [7], and model cascades [2], further reduce generation cost. These methods provide different speed–quality tradeoffs. Our work focuses on the four common fidelity knobs above because they can be adjusted online at chunk boundaries and expose practical latency–quality tradeoffs for real-time AR-DiT serving.

9 Conclusion and Future Work

AR-DiT-based real-time video generation shifts the serving objective from completing a request quickly to keeping each stream ahead of its playout timeline. This paper presents SlackServe, a playout-slack-driven serving system for AR-DiT-based real-time video generation. SlackServe uses service credit to guide both cross-stream resource reallocation

and within-stream fidelity selection. Across streams, it redirects resources to urgent streams through three-tier priority queues, re-homing, and elastic SP. Within each stream, BMPR selects Pareto-optimal fidelity configurations under a playout-slack budget and a quality floor.

As future work, we plan to extend SlackServe to multi-task and multi-resolution serving, where streams may have different generation objectives, resolutions, and playout rates. While SlackServe is, to our knowledge, the first step toward playout-slack-driven serving for streaming video generation, it still relies on profiled frontiers and heuristic control policies, which we plan to refine.

References

[1] Amey Agrawal, Nitin Kedia, Ashish Panwar, Jayashree Mohan, Nipun Kwatra, Bhargav Gulavani, Alexey Tumanov, and Ramachandran Ramjee. Taming {Throughput-Latency} tradeoff in {LLM} inference with {Sarathi-Serve}. In *18th USENIX symposium on operating systems design and implementation (OSDI 24)*, pages 117–134, 2024.

[2] Sohaib Ahmad, Qizheng Yang, Haoliang Wang, Ramesh K Sitaraman, and Hui Guan. Diffserve: Efficiently serving text-to-image diffusion models with query-aware model scaling. *Proceedings of Machine Learning and Systems*, 7, 2025.

[3] AI-Dynamo. Nvidia inference xfer library (nixl). GitHub repository, 2025. https://github.com/ai-dynamo/nixl.

[4] Athula Balachandran, Vyas Sekar, Aditya Akella, Srinivasan Seshan, Ion Stoica, and Hui Zhang. A quest for an internet video quality-of-experience metric. In *Proceedings of the 11th ACM workshop on hot topics in networks*, pages 97–102, 2012.

[5] Betsy Beyer, Chris Jones, Jennifer Petoff, and Niall Richard Murphy. *Site Reliability Engineering: How Google Runs Production Systems*. O'Reilly Media, 2016.

[6] Andreas Blattmann, Tim Dockhorn, Sumith Kulal, Daniel Mendelevitch, Maciej Kilian, Dominik Lorenz, Yam Levi, Zion English, Vikram Voleti, Adam Letts, et al. Stable video diffusion: Scaling latent video diffusion models to large datasets. *arXiv preprint arXiv:2311.15127*, 2023.

[7] Daniel Bolya and Judy Hoffman. Token merging for fast stable diffusion. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 4599–4603, 2023.

[8] Tom Brown, Benjamin Mann, Nick Ryder, Melanie Subbiah, Jared D Kaplan, Prafulla Dhariwal, Arvind Neelakantan, Pranav Shyam, Girish Sastry, Amanda Askell, et al. Language models are few-shot learners. *Advances in neural information processing systems*, 33:1877–1901, 2020.

[9] Haoxin Chen, Yong Zhang, Xiaodong Cun, Menghan Xia, Xintao Wang, Chao Weng, and Ying Shan. Videocrafter2: Overcoming data limitations for high-quality video diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 7310–7320, 2024.

[10] Junyi Chen, Chuheng Du, Renyuan Liu, Shuochao Yao, Dingtian Yan, Jiang Liao, Shengzhong Liu, Fan Wu, and Guihai Chen. Tokenflow: Responsive llm text streaming serving under request burst via preemptive scheduling. In *Proceedings of the 21st European Conference on Computer Systems*, pages 497–513, 2026.

[11] Pengtao Chen, Xianfang Zeng, Maosen Zhao, Mingzhu Shen, Wei Cheng, Gang Yu, and Tao Chen. Sparse-vdit: Unleashing the power of sparse attention to accelerate video diffusion transformers. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 40, pages 2957–2965, 2026.

[12] Justin Cui, Jie Wu, Ming Li, Tao Yang, Xiaojie Li, Rui Wang, Andrew Bai, Yuanhao Ban, and Cho-Jui Hsieh. Self-forcing++: Towards minute-scale high-quality video generation. *arXiv preprint arXiv:2510.02283*, 2025.

[13] Zhenbang Du, Yonggan Fu, Lifu Wang, Jiayi Qian, Xiao Luo, and Yingyan Celine Lin. Fewer denoising steps or cheaper per-step inference: Towards compute-optimal diffusion model deployment. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 3001–3010, 2025.

[14] Tianrui Feng, Zhi Li, Shuo Yang, Haocheng Xi, Muyang Li, Xiuyu Li, Lvmin Zhang, Keting Yang, Kelly Peng, Song Han, et al. Streamdiffusionv2: A streaming system for dynamic and interactive video generation. *arXiv preprint arXiv:2511.07399*, 2025.

[15] Kaifeng Gao, Jiaxin Shi, Hanwang Zhang, Chunping Wang, Jun Xiao, and Long Chen. Ca2-vdm: Efficient autoregressive video diffusion model with causal generation and cache sharing. *arXiv preprint arXiv:2411.16375*, 2024.

[16] Matthew Halpern, Behzad Boroujerdian, Todd Mummert, Evelyn Duesterwald, and Vijay Janapa Reddi. One size does not fit all: Quantifying and exposing the accuracy-latency trade-off in machine learning cloud service apis via tolerance tiers. *arXiv preprint arXiv:1906.11307*, 2019.

[17] Roberto Henschel, Levon Khachatryan, Hayk Poghosyan, Daniil Hayrapetyan, Vahram Tadevosyan, Zhangyang Wang, Shant Navasardyan, and Humphrey Shi. Streamingt2v: Consistent, dynamic, and extendable long video generation from text. In *Proceedings of the Computer Vision and Pattern Recognition Conference*, pages 2568–2577, 2025.

[18] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020.

[19] Te-Yuan Huang, Ramesh Johari, Nick McKeown, Matthew Trunnell, and Mark Watson. A buffer-based approach to rate adaptation: Evidence from a large video streaming service. In *Proceedings of the 2014 ACM conference on SIGCOMM*, pages 187–198, 2014.

[20] Xun Huang, Zhengqi Li, Guande He, Mingyuan Zhou, and Eli Shechtman. Self forcing: Bridging the train-test gap in autoregressive video diffusion. *Advances in Neural Information Processing Systems*, 38:167283–167308, 2026.

[21] Yanping Huang, Youlong Cheng, Ankur Bapna, Orhan Firat, Dehao Chen, Mia Chen, HyoukJoong Lee, Jiquan Ngiam, Quoc V Le, Yonghui Wu, et al. Gpipe: Efficient training of giant neural networks using pipeline parallelism. *Advances in neural information processing systems*, 32, 2019.

[22] Ziqi Huang, Yinan He, Jiashuo Yu, Fan Zhang, Chenyang Si, Yuming Jiang, Yuanhan Zhang, Tianxing Wu, Qingyang Jin, Nattapol Chanpaisit, Yaohui Wang, Xinyuan Chen, Limin Wang, Dahua Lin, Yu Qiao, and Ziwei Liu. VBench: Comprehensive benchmark suite for video generative models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2024.

[23] Sam Ade Jacobs, Masahiro Tanaka, Chengming Zhang, Minjia Zhang, Shuaiwen Leon Song, Samyam Rajbhandari, and Yuxiong He. Deepspeed ulysses: System optimizations for enabling training of extreme long sequence transformer models. *arXiv preprint arXiv:2309.14509*, 2023.

[24] Kumara Kahatapitiya, Haozhe Liu, Sen He, Ding Liu, Menglin Jia, Chenyang Zhang, Michael S Ryoo, and Tian Xie. Adaptive caching for faster video generation with diffusion transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 15240–15252, 2025.

[25] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. *Advances in neural information processing systems*, 35:26565–26577, 2022.

[26] Akio Kodaira, Tingbo Hou, Ji Hou, Markos Georgopoulos, Felix Juefei-Xu, Masayoshi Tomizuka, and Yue Zhao. Streamdit: Real-time streaming text-to-video generation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 29200–29210, 2026.

[27] Akio Kodaira, Chenfeng Xu, Toshiki Hazama, Takanori Yoshimoto, Kohei Ohno, Shogo Mitsuhori, Soichi Sugano, Hanying Cho, Zhijian Liu, Masayoshi Tomizuka, et al. Streamdiffusion: A pipeline-level solution for real-time interactive generation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 12371–12380, 2025.

[28] Weijie Kong, Qi Tian, Zijian Zhang, Rox Min, Zuozhuo Dai, Jin Zhou, Jiangfeng Xiong, Xin Li, Bo Wu, Jianwei Zhang, et al. Hunyuanvideo: A systematic framework for large video generative models. *arXiv preprint arXiv:2412.03603*, 2024.

[29] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. Efficient memory management for large language model serving with pagedattention. In *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*, 2023.

[30] Zeqi Lai, Y Charlie Hu, Yong Cui, Linhui Sun, and Ningwei Dai. Furion: Engineering high-quality immersive virtual reality on today's mobile devices. In *Proceedings of the 23rd Annual International Conference on Mobile Computing and Networking*, pages 409–421, 2017.

[31] Haodong Li, Shaoteng Liu, Zhe Lin, and Manmohan Chandraker. Rolling sink: Bridging limited-horizon training and open-ended testing in autoregressive video diffusion. *arXiv preprint arXiv:2602.07775*, 2026.

[32] Suyi Li, Lingyun Yang, Xiaoxiao Jiang, Hanfeng Lu, Dakai An, Zhipeng Di, Weiyi Lu, Jiawei Chen, Kan Liu, Yinghao Yu, et al. Katz: Efficient workflow serving for diffusion models with many adapters. In *2025 USENIX Annual Technical Conference (USENIX ATC 25)*, pages 1037–1052, 2025.

[33] Ji Lin, Jiaming Tang, Haotian Tang, Shang Yang, Wei-Ming Chen, Wei-Chen Wang, Guangxuan Xiao, Xingyu Dang, Chuang Gan, and Song Han. Awq: Activation-aware weight quantization for on-device llm compression and acceleration. *Proceedings of machine learning and systems*, 6:87–100, 2024.

[34] Hao Liu, Matei Zaharia, and Pieter Abbeel. Ring attention with blockwise transformers for near-infinite context. *arXiv preprint arXiv:2310.01889*, 2023.

[35] Jiachen Liu, Jae-Won Chung, Zhiyu Wu, Fan Lai, Myungjin Lee, and Mosharaf Chowdhury. Andes: Defining and enhancing quality-of-experience in llm-based text streaming services. *arXiv preprint arXiv:2404.16283*, 2024.

[36] Kunhao Liu, Wenbo Hu, Jiale Xu, Ying Shan, and Shijian Lu. Rolling forcing: Autoregressive long video diffusion in real time. *arXiv preprint arXiv:2509.25161*, 2025.

[37] Cheng Lu, Yuhao Zhou, Fan Bao, Jianfei Chen, Chongxuan Li, and Jun Zhu. Dpm-solver++: Fast solver for guided sampling of diffusion probabilistic models. *Machine Intelligence Research*, 22(4):730–751, 2025.

[38] Runyu Lu, Shiqi He, Wenxuan Tan, Shenggui Li, Ruofan Wu, Jeff J Ma, Ang Chen, and Mosharaf Chowdhury. Tetriserve: Efficient dit serving for heterogeneous image generation. *arXiv preprint arXiv:2510.01565*, 2025.

[39] Chengtao Lv, Yumeng Shi, Yushi Huang, Ruihao Gong, Shen Ren, and Wenya Wang. Light forcing: Accelerating autoregressive video diffusion via sparse attention. *arXiv preprint arXiv:2602.04789*, 2026.

[40] Hongzi Mao, Ravi Netravali, and Mohammad Alizadeh. Neural adaptive video streaming with pensieve. In *Proceedings of the conference of the ACM special interest group on data communication*, pages 197–210, 2017.

[41] Kaisa Miettinen. *Nonlinear multiobjective optimization*, volume 12. Springer Science & Business Media, 1999.

[42] Philipp Moritz, Robert Nishihara, Stephanie Wang, Alexey Tumanov, Richard Liaw, Eric Liang, Melih Elibol, Zongheng Yang, William Paul, Michael I. Jordan, and Ion Stoica. Ray: A distributed framework for emerging ai applications. In *Proceedings of the Symposium on Operating Systems Design and Implementation (OSDI)*, 2018.

[43] Deepak Narayanan, Aaron Harlap, Amar Phanishayee, Vivek Seshadri, Nikhil R Devanur, Gregory R Ganger, Phillip B Gibbons, and Matei Zaharia. Pipedream: Generalized pipeline parallelism for dnn training. In *Proceedings of the 27th ACM symposium on operating systems principles*, pages 1–15, 2019.

[44] NVIDIA. TensorRT-LLM. https://github.com/NVIDIA/TensorRT-LLM, 2023. An open-source library for optimizing and deploying large language model inference on NVIDIA GPUs.

[45] NVIDIA. CUDA C++ Programming Guide. https://docs.nvidia.com/cuda/cuda-c-programming-guide/, 2026. NVIDIA CUDA documentation.

[46] NVIDIA. GPUDirect RDMA. https://docs.nvidia.com/cuda/gpudirect-rdma/, 2026. NVIDIA GPUDirect RDMA documentation.

[47] NVIDIA. NVIDIA H100 Tensor Core GPU. https://www.nvidia.com/en-us/data-center/h100/, 2026. Product specifications.

[48] NVIDIA. NVIDIA NVLink. https://www.nvidia.com/en-us/data-center/nvlink/, 2026. NVIDIA NVLink product documentation.

[49] NVIDIA. NVIDIA Quantum-2 InfiniBand Platform. https://www.nvidia.com/en-us/networking/quantum2/, 2026. NDR 400Gb/s InfiniBand platform specifications.

[50] NVIDIA Corporation. NVIDIA Collective Communication Library (NCCL). https://developer.nvidia.com/nccl, 2025. NCCL 2.x documentation, accessed 2025-08-19.

[51] William Peebles and Saining Xie. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF international conference on computer vision*, pages 4195–4205, 2023.

[52] Python Software Foundation. asyncio — asynchronous I/O. https://docs.python.org/3/library/asyncio.html, 2026. Python standard library documentation.

[53] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of machine learning research*, 21(140):1–67, 2020.

[54] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 10684–10695, 2022.

[55] Sheldon M Ross. *Stochastic processes*. John Wiley & Sons, 1995.

[56] Jay Shah, Ganesh Bikshandi, Ying Zhang, Vijay Thakkar, Pradeep Ramani, and Tri Dao. Flashattention-3: Fast and accurate attention with asynchrony and low-precision. *Advances in Neural Information Processing Systems*, 37:68658–68685, 2024.

[57] Joonghyuk Shin, Zhengqi Li, Richard Zhang, Jun-Yan Zhu, Jaesik Park, Eli Shechtman, and Xun Huang. Motionstream: Real-time video generation with interactive motion controls. *arXiv preprint arXiv:2511.01266*, 2025.

[58] Mohammad Shoeybi, Mostofa Patwary, Raul Puri, Patrick LeGresley, Jared Casper, and Bryan Catanzaro. Megatron-lm: Training multi-billion parameter language models using model parallelism. *arXiv preprint arXiv:1909.08053*, 2019.

[59] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. *arXiv preprint arXiv:2010.02502*, 2020.

[60] Desen Sun, Henry Tian, Tim Lu, and Sihang Liu. Flexcache: Flexible approximate cache system for video diffusion. *arXiv preprint arXiv:2501.04012*, 2024.

[61] Hansi Teng, Hongyu Jia, Lei Sun, Lingzhi Li, Maolin Li, Mingqiu Tang, Shuai Han, Tianning Zhang, WQ Zhang, Weifeng Luo, et al. Magi-1: Autoregressive video generation at scale. *arXiv preprint arXiv:2505.13211*, 2025.

[62] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. *Advances in neural information processing systems*, 30, 2017.

[63] Team Wan, Ang Wang, Baole Ai, Bin Wen, Chaojie Mao, Chen-Wei Xie, Di Chen, Feiwu Yu, Haiming Zhao, Jianxiao Yang, Jianyuan Zeng, Jiayu Wang, Jingfeng Zhang, Jingren Zhou, Jinkai Wang, Jixuan Chen, Kai Zhu, Kang Zhao, Keyu Yan, Lianghua Huang, Mengyang Feng, Ningyi Zhang, Pandeng Li, Pingyu Wu, Ruihang Chu, Ruili Feng, Shiwei Zhang, Siyang Sun, Tao Fang, Tianxing Wang, Tianyi Gui, Tingyu Weng, Tong Shen, Wei Lin, Wei Wang, Wei Wang, Wenmeng Zhou, Wente Wang, Wenting Shen, Wenyuan Yu, Xianzhong Shi, Xiaoming Huang, Xin Xu, Yan Kou, Yangyu Lv, Yifei Li, Yijing Liu, Yiming Wang, Yingya Zhang, Yitong Huang, Yong Li, You Wu, Yu Liu, Yulin Pan, Yun Zheng, Yuntao Hong, Yupeng Shi, Yutong Feng, Zeyinzi Jiang, Zhen Han, Zhi-Fan Wu, and Ziyu Liu. Wan: Open and advanced large-scale video generative models. *arXiv preprint arXiv:2503.20314*, 2025.

[64] Xiang Wang, Shiwei Zhang, Han Zhang, Yu Liu, Yingya Zhang, Changxin Gao, and Nong Sang. Videolcm: Video latent consistency model. *arXiv preprint arXiv:2312.09109*, 2023.

[65] Yaohui Wang, Xinyuan Chen, Xin Ma, Shangchen Zhou, Ziqi Huang, Yi Wang, Ceyuan Yang, Yinan He, Jiashuo Yu, Peiqing Yang, et al. Lavie: High-quality video generation with cascaded latent diffusion models. *International Journal of Computer Vision*, 133(5):3059–3078, 2025.

[66] Bingyang Wu, Shengyu Liu, Yinmin Zhong, Peng Sun, Xuanzhe Liu, and Xin Jin. Loongserve: Efficiently serving long-context large language models with elastic sequence parallelism. In *Proceedings of the ACM SIGOPS 30th Symposium on Operating Systems Principles*, pages 640–654, 2024.

[67] Haocheng Xi, Shuo Yang, Yilong Zhao, Chenfeng Xu, Muyang Li, Xiuyu Li, Yujun Lin, Han Cai, Jintao Zhang, Dacheng Li, et al. Sparse video-gen: Accelerating video diffusion transformers with spatial-temporal sparsity. In *Forty-second International Conference on Machine Learning*.

[68] Yifei Xia, Fangcheng Fu, Hao Yuan, Hanke Zhang, Xupeng Miao, Yijun Liu, Suhan Ling, Jie Jiang, and Bin Cui. Tridentserve: A stage-level serving system for diffusion pipelines. *arXiv preprint arXiv:2510.02838*, 2025.

[69] Yifei Xia, Suhan Ling, Fangcheng Fu, Yujie Wang, Huixia Li, Xuefeng Xiao, and Bin Cui. Training-free and adaptive sparse attention for efficient long video generation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 15982–15993, 2025.

[70] Guangxuan Xiao, Yuandong Tian, Beidi Chen, Song Han, and Mike Lewis. Efficient streaming language models with attention sinks. In *International Conference on Learning Representations*, volume 2024, pages 21875–21895, 2024.

[71] Zhuoyi Yang, Jiayan Teng, Wendi Zheng, Ming Ding, Shiyu Huang, Jiazheng Xu, Yuanming Yang, Wenyi Hong, Xiaohan Zhang, Guanyu Feng, et al. Cogvideox: Text-to-video diffusion models with an expert transformer. *arXiv preprint arXiv:2408.06072*, 2024.

[72] Tianwei Yin, Michaël Gharbi, Richard Zhang, Eli Shechtman, Fredo Durand, William T Freeman, and Taesung Park. One-step diffusion with distribution matching distillation. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 6613–6623, 2024.

[73] Tianwei Yin, Qiang Zhang, Richard Zhang, William T Freeman, Fredo Durand, Eli Shechtman, and Xun Huang. From slow bidirectional to fast autoregressive video diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 22963–22974, 2025.

[74] Xiaoqi Yin, Abhishek Jindal, Vyas Sekar, and Bruno Sinopoli. A control-theoretic approach for dynamic adaptive video streaming over http. In *Proceedings of the 2015 ACM conference on special interest group on data communication*, pages 325–338, 2015.

[75] Gyeong-In Yu, Joo Seong Jeong, Geon-Woo Kim, Soojeong Kim, and Byung-Gon Chun. Orca: A distributed serving system for {Transformer-Based} generative models. In *16th USENIX symposium on operating systems design and implementation (OSDI 22)*, pages 521–538, 2022.

[76] Shenghai Yuan, Yuanyang Yin, Zongjian Li, Xinwei Huang, Xiao Yang, and Li Yuan. Helios: Real real-time long video generation model. *arXiv preprint arXiv:2603.04379*, 2026.

[77] Jintao Zhang, Haofeng Huang, Pengle Zhang, Jia Wei, Jun Zhu, and Jianfei Chen. Sageattention2: Efficient attention with thorough outlier smoothing and per-thread int4 quantization. *arXiv preprint arXiv:2411.10958*, 2024.

[78] Jintao Zhang, Pengle Zhang, Jun Zhu, Jianfei Chen, et al. Sageattention: Accurate 8-bit attention for plug-and-play inference acceleration. In *International Conference on Learning Representations*, volume 2025, pages 71566–71585, 2025.

[79] Yinmin Zhong, Shengyu Liu, Junda Chen, Jianbo Hu, Yibo Zhu, Xuanzhe Liu, Xin Jin, and Hao Zhang. {DistServe}: Disaggregating prefill and decoding for goodput-optimized large language model serving. In *18th USENIX Symposium on Operating Systems Design and Implementation (OSDI 24)*, pages 193–210, 2024.

[80] Hongzhou Zhu, Min Zhao, Guande He, Hang Su, Chongxuan Li, and Jun Zhu. Causal forcing: Autoregressive diffusion distillation done right for high-quality real-time interactive video generation. *arXiv preprint arXiv:2602.02214*, 2026.

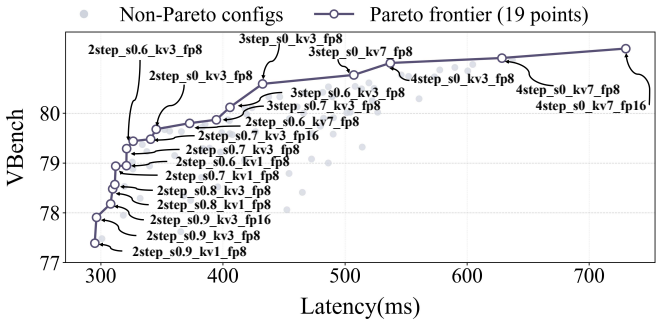

Figure 19. Pareto frontier for Causal-Forcing.

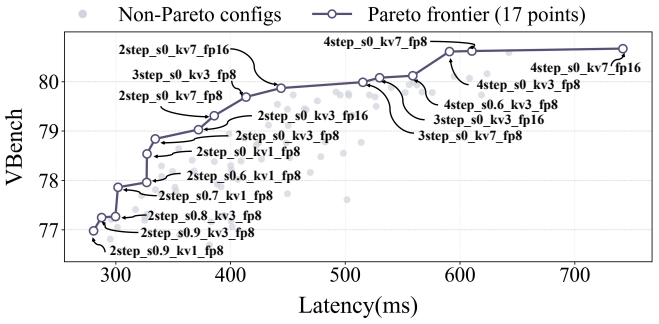

Figure 20. Pareto frontier for Self-Forcing.

A Fidelity-Configuration Space and Pareto Frontier in BMPR

This appendix details the candidate fidelity-configuration space and the Pareto frontier used by Bi-Modal Pareto Routing (§5).

Fidelity-configuration space. SlackServe exposes the four fidelity knobs of §2.1: denoising steps $S \in \{2, 3, 4\}$, attention sparsity $\rho \in \{0, 0.6, 0.7, 0.8, 0.9\}$, KV-window size $W \in \{1, 3, 7\}$ chunks, and quantization mode $Q \in \{\text{FP16}, \text{FP8}\}$. Their Cartesian product yields $3 \times 5 \times 3 \times 2 = 90$ candidate fidelity configurations $\text{cfg} = (S, \rho, W, Q)$, with $(4, 0, 7, \text{FP16})$ as the highest-quality reference.

Pareto frontier and quality floor. We profile every candidate's fidelity configuration offline over the VBench prompt set. For each fidelity configuration, we measure its average per-chunk generation latency L in ms/chunk and obtain its quality score using VBench. From the profiled pairs, SlackServe discards every fidelity configuration dominated in the (latency, quality) plane (§5.2) and keeps the non-dominated set as the empirical Pareto frontier F. Figure 19 and Figure 20 show the Pareto frontiers of Causal-Forcing and Self-Forcing, together with the median quality value Q_{floor}. BMPR routes only within F and never selects a fidelity configuration whose quality is below Q_{floor}.

B Workload Details

This appendix details the five workloads used in §7.2. Unless otherwise noted, all workloads share the same per-stream settings, with prompts drawn from VBench, which contains 946 samples. Each stream's target length is sampled uniformly from $\{81, 129, 161, 241\}$ frames, corresponding to about 5–15 s at 16 fps. Video is generated at 480p with three latent frames per chunk, and the playout rate is 16 fps. Here stream lengths are counted in pixel-space frames, whereas a chunk is defined in latent frames: the two are related by the VAE temporal compression factor, which is about 4× for the AR-DiTs we use, so a chunk of three latent frames corresponds to roughly twelve pixel frames (about 0.75 s of playout at 16 fps).

Steady. Steady is the default workload. Streams arrive according to a Poisson process with rate $\lambda = 1$ stream/s, yielding 946 streams per run. Steady measures system behavior under a stationary load and serves as the base arrival process for the other synthetic workloads.

Burst. Burst extends Steady by introducing synchronized arrivals while preserving the total number of streams. We fix the number of burst points to three and place them at the 20%, 50%, and 80% progress points of the Steady arrival sequence. At each burst point, 10% of all streams are reassigned to arrive simultaneously at that timestamp. This construction keeps the total number of streams unchanged while creating short-lived overloads that model flash crowds, hot events, or batches of users joining at once. The resulting transient load spikes stress cross-worker re-homing (§4.2) and test whether the system can quickly redistribute work after synchronized arrivals.

Prompt-switch. Prompt-switch extends Steady by injecting prompt-switch signals into each stream. Streams with target lengths of 81 frames receive one switch, streams with target lengths of 129 or 161 frames receive two switches, and streams with target lengths of 241 frames receive three switches. The switch positions are sampled uniformly at random within each stream. A prompt switch represents a user changing the generation condition and then continuing generation under the new condition; chunks buffered under the old condition are no longer useful, so the stream's playout slack is reset to the initial TTFC. To preserve the semantic continuity required by VBench evaluation, we do not actually change the text prompt. Instead, we inject a playout-slack reset signal that captures the scheduling effect of a prompt switch without altering the generated content. This workload creates isolated low-slack regions within otherwise normal streams, stressing per-chunk fidelity selection (§5) and elastic SP (§4.3).

Algorithm 2 High-Level Workflow of SlackServe

Input: stream requests R, cluster of workers C

State: per-stream slack, service credit, fidelity configuration, and per-worker priority queues
while system runs **do**
  if a new stream request s arrives **then**
    $home \leftarrow \text{ChooseHomeWorker}(s, C)$
    $s.slack_{\text{init}} \leftarrow 4 \times \text{EstimatedFirstChunkTime}(s)$
    $\text{Admit}(s, home)$
  else if a control tick fires **then**
    for each active stream u **do**
      $u.cfg_{\text{next}} \leftarrow \text{BMPRSelect}(u)$
      $u.credit \leftarrow \text{ComputeCredit}(u, u.cfg_{\text{next}})$
      $u.tier \leftarrow \text{ClassifyTier}(u.credit, u.cfg_{\text{next}})$
    $\text{UpdatePriorityQueues}(streams, C)$
    $P_{\text{re}} \leftarrow \text{Rehoming}(streams, C)$
    $P_{\text{sp}} \leftarrow \text{ElasticSP}(streams, C)$
    $\text{Async}(\text{StatePlane.Transfer}(P_{\text{re}}, P_{\text{sp}}))$
  else if a step or chunk boundary occurs on worker w **then**
    $\text{ApplyDecision}(w)$
    if the boundary is a chunk boundary **then**
      $\text{ApplyNextFidelityConfig}(w)$
    $\text{DispatchNext}(w.priority\_queue)$

Pause. Pause extends Steady by injecting client-side pause signals into each stream. Streams with target lengths of 81 frames receive one pause, streams with target lengths of 129 or 161 frames receive two pauses, and streams with target lengths of 241 frames receive three pauses. Pause positions are sampled uniformly at random within each stream. Each pause lasts for 20% of the stream's target duration. During a pause, client-side playout halts while generation may continue, allowing playout slack to accumulate. This workload models user behaviors such as temporary viewing pauses or interactions that delay consumption. Compared with the other synthetic workloads, Pause is less adversarial in terms of instantaneous load, but it tests whether SlackServe can preserve and exploit accumulated slack rather than spending it uniformly.

Proprietary trace. The proprietary trace is a realistic enterprise trace with mixed arrival patterns, including interleaved steady periods, bursts, and idle gaps. We use its arrival pattern as the external workload shape and uniformly subsample the trace arrival points to match our cluster scale. To make the trace compatible with our video-generation benchmark, we replace the original prompts and target lengths with VBench prompts and sample each stream's target length uniformly from $\{81, 129, 161, 241\}$ frames. This workload captures more complex real-world dynamics than the synthetic traces, combining non-stationary arrivals with heterogeneous stream lengths.

C Additional Implementation Details

This appendix provides additional implementation details of SlackServe's control loop, re-homing policy, elastic sequence parallelism, and sequence-parallel KV-state transfer. These details complement the design described in §3 and §4.

C.1 High-Level Control Workflow

Algorithm 2 summarizes the high-level workflow of SlackServe. The system is event-driven. New stream arrivals are handled by admission control, periodic control ticks update fidelity and resource-allocation decisions, and step/chunk boundaries serve as safe points where published decisions take effect.

At each control tick, SlackServe first selects the next-chunk fidelity configuration for each active stream using BMPR. It then recomputes service credit under the selected configuration and updates the stream's urgency tier. Based on the updated tiers, the controller generates a bounded re-homing plan and an elastic-SP plan, composes these decisions, and publishes them to the execution and state planes. State transfers are issued asynchronously and do not block the control loop. At safe boundaries, a worker applies the latest published decision and dispatches the next stream from its priority queue. Fidelity change and re-homing are applied only at chunk boundaries, while priority preemption and elastic SP decisions may take effect at step or chunk boundaries.

C.2 Details of Re-homing

Re-homing is intentionally conservative. Although moving an urgent stream to a slack-rich worker can reduce stalls, frequent migration can consume bandwidth, increase KV-transfer pressure, and cause oscillation. SlackServe therefore uses three safeguards.

First, each migrated stream enters a cooldown period of 60 seconds. A stream in cooldown is not eligible for another re-homing decision, even if it later becomes URGENT again. This prevents repeated back-and-forth movement of the same stream under transient slack fluctuations.

Second, SlackServe bounds the number of migrations generated at each control tick. Each sender worker can migrate out at most two streams per tick, and each receiver worker can accept at most one migrated stream per tick. These caps limit transfer bursts and keep asynchronous KV migration from interfering with foreground chunk generation.

Third, SlackServe prefers intra-node migration before cross-node migration. Cross-node re-homing is used only when no intra-node receiver is available and the sender remains URGENT-heavy. In our default configuration, the re-homing planner selects receivers with no URGENT or NORMAL streams, so that migrated streams are placed only on workers with sufficient slack headroom. These constraints make re-homing stable and reduce migration-induced oscillation.

C.3 Elastic Sequence Parallelism Policy

SlackServe pre-initializes all candidate intra-node SP2 groups before serving, so elastic SP does not create communication groups at runtime. Elastic SP is used only as a last-resort recovery mechanism for streams whose service credit is negative. In the default configuration, SlackServe restricts elastic SP to an SP degree of at most two. That is, an urgent stream can borrow at most one donor worker. The donor must be a RELAXED worker in the same node as the urgent stream's current worker. If no such donor exists, elastic SP is not triggered for that stream.

This conservative policy is motivated by two practical considerations. First, larger SP degrees provide diminishing returns for the chunk sizes used in our streaming setting, because the additional collective-communication overhead can offset the reduced per-GPU computation. Second, cross-node elastic SP introduces substantially higher communication cost and more complex state synchronization. Restricting elastic SP to intra-node SP2 keeps the mechanism predictable and prevents recovery actions from degrading cluster-wide throughput.

When a donor is selected, SlackServe switches the urgent stream to the corresponding pre-initialized intra-node SP2 group. The donor is released at the next safe boundary once the stream's service credit recovers to the NORMAL tier, after which the stream switches back to its default SP1 execution group. Donor selection is credit-aware: SlackServe chooses the highest-credit RELAXED worker in the same node, so that borrowing compute is least likely to push the donor's local streams toward future stalls.

C.4 Sequence-Parallel KV-State Transfer

SlackServe implements sequence parallelism following the Ulysses-style partitioning strategy, where attention heads are partitioned across workers in the SP group. This partitioning also determines how KV state is redistributed during elastic SP. When a stream switches from SP1 to a pre-initialized SP2 group, SlackServe migrates only the KV pages corresponding to the head partition assigned to the donor worker.

This head-partition-aware transfer avoids an additional all-to-all redistribution of the KV cache. The State Plane directly transfers the required paged KV ranges to the destination worker according to the target SP layout.

D Control and State-Plane Overheads

This appendix evaluates the overhead introduced by SlackServe's control and state planes. We focus on three questions: (1) whether the global controller can make decisions fast enough as the number of active streams increases, (2) how long KV-state transfers take when resource reallocation triggers state movement, and (3) how much of the transfer time remains on the critical path after asynchronous layer-wise streaming.

D.1 Controller Scalability

At each control tick, the Control Plane updates stream slack, selects per-chunk fidelity configurations, reorders per-worker queues, and generates re-homing and elastic-SP decisions. Let U be the number of active streams, G the number of workers, and $|F|$ the size of the profiled Pareto frontier used by BMPR. The controller performs four main operations. First, it computes service credits for all streams in $O(U)$. Second, BMPR selects the next fidelity configuration for each stream by scanning the Pareto frontier, which costs $O(U|F|)$. In our implementation, $|F|$ is small and fixed after offline profiling. Third, three-tier queue construction and within-tier ordering cost $O\left(\sum_{g=1}^{G} U_g \log U_g\right)$, where U_g is the number of streams assigned to worker g.

Table 4. *Control Plane* scalability on the 16-GPU testbed. We report the average end-to-end time of one control tick, including slack update, fidelity selection, and resource-reallocation planning.

# Active streams	Avg. control time (ms)	Fraction of 3s tick
64	9.1	0.30%
128	10.8	0.36%
256	13.7	0.46%
512	20.4	0.68%
1024	39.6	1.32%

Table 5. *State Plane* overheads on the Steady workload. KV-transfer latency is mostly hidden by asynchronous layer-wise streaming.

(a) KV transfer time

Time range (ms)	Events	Fraction
0–5	8	6.7%
5–10	18	15.0%
10–15	24	20.0%
15–20	21	17.5%
20–30	17	14.2%
30–40	11	9.2%
40–60	8	6.7%
60–80	5	4.2%
80–120	5	4.2%
120+	3	2.5%
Avg.		31.8 ms
P95		118.4 ms

(b) Residual dispatch wait

Time range (ms)	Events	Fraction
0–5	82	68.3%
5–10	22	18.3%
10–15	9	7.5%
15–20	4	3.3%
20–25	2	1.7%
25+	1	0.8%
Avg.		4.4 ms
P95		16.6 ms

Fourth, re-homing and elastic-SP planning operate over workers and urgent streams. With bounded per-tick migration caps, their cost is linear in the number of candidate urgent streams plus $O(G^2)$ for sender–receiver matching. Overall, for a fixed worker count, the controller scales near-linearly with the number of active streams.

We measure the average end-to-end controller latency on the same 16-GPU testbed used in the main evaluation. To stress the controller without changing the GPU cluster size, we replay controller states with 64–1024 active streams and keep the same 3-second control interval as in the main system. Table 4 shows that even with 1024 active streams, one control tick takes 39.6 ms on average, which is only 1.32% of the default control interval. This overhead is small compared with chunk-generation time and does not affect the critical path of video generation.

D.2 State-Plane Transfer Overhead

SlackServe moves KV state when re-homing or elastic SP changes the worker set of a stream. We measure all completed KV transfers on the Steady workload. Table 5 summarizes both the raw KV-transfer latency and the residual waiting time observed by affected dispatches.

Most transfers complete within tens of milliseconds because SlackServe prefers intra-node re-homing and issues transfers asynchronously. The tail comes from cross-node transfers, larger KV windows, and runtime overheads such as page lookup, transfer submission, CUDA-event synchronization, and layer-wise transfer scheduling. Across all transfer events, the average transfer time is 31.8 ms and the P95 transfer time is 118.4 ms.

Residual waiting is much smaller than raw transfer latency. A migrating stream is reinserted once its first-layer KV pages are ready, while later layers continue transferring in the background. As a result, most affected dispatches wait for less than 5 ms. The average residual wait is 4.4 ms, and the P95 residual wait is 16.6 ms. Only 13.8% of the average transfer latency remains on the critical path.